



API REST Products

Recordando un poco, NEST se organiza en **módulos** que integran los controladores o **services** pertenecientes entre si, es indispensable crear los elementos relacionados con los controladores, services, module para poder manejar las peticiones.

Sin embargo, cuando se quiere incluir el uso de elementos como objetos (Productos, Categorías, usuarios, etc.) se requiere de una clase que indique la forma del objeto, es decir, como se crea, que propiedades tiene dicho objeto, para que en las peticiones y respuestas se pueda controlar el flujo de la información.

A esto mencionado anteriormente se le conoce como, DTO. Un **Data Transfer Object** por sus siglas en ingles **DTO**, es un objeto que se utiliza para transferir información entre diferentes partes de la aplicación. Son especialmente útiles para **definir** la forma y estructura de los datos que se espera recibir o enviar.

En este material se incluirá el uso del DTO para definir la API

Comencemos....

Creando el Proyecto

Crear un proyecto con el nombre: **api-products**, recordar que el proyecto se crea mediante el comando **nest new <nombreProyecto>**

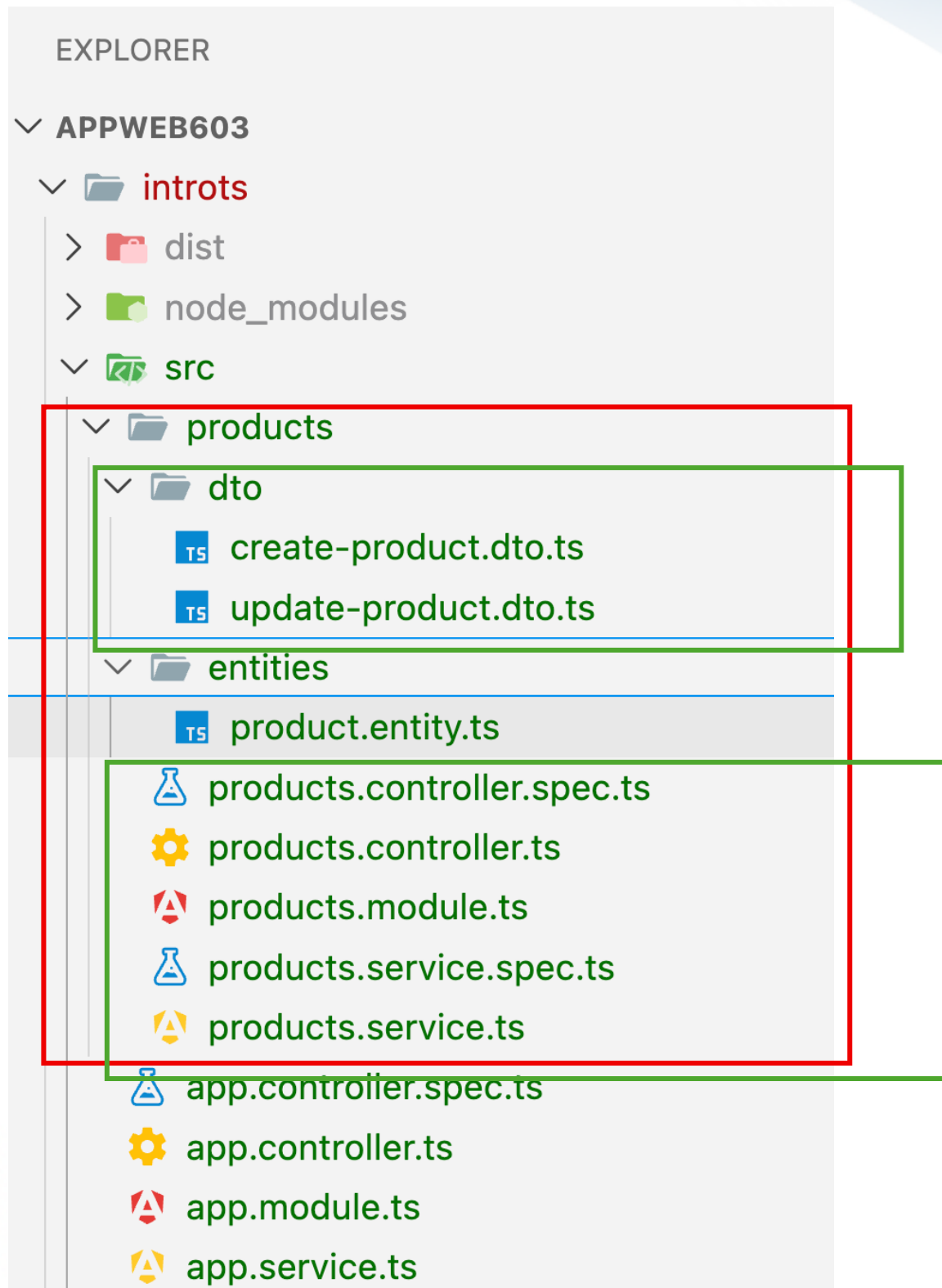
Agregando elemento producto

En este caso, se agregará los elementos module, service, controller y el DTO mediante un comando que nos genera toda la estructura.

- 1.- Ejecutar el comando: **nest g resource**, este comando creará la estructura completa.
- 2.- Elegir el nombre, en este caso es: **products**
- 3.- Elegir el tipo de protocolo para transferir información, en este caso: **REST API**
- 4.- Preguntará si desean crear el CRUD, seleccionar **yes**

```
ingepollo@MacBook-Air-de-Luis: ~ % nest g resource
✔ What name would you like to use for this resource (plural, e.g., "users")? products
✔ What transport layer do you use? REST API
✔ Would you like to generate CRUD entry points? yes
CREATE src/products/products.controller.spec.ts (596 bytes)
CREATE src/products/products.controller.ts (957 bytes)
CREATE src/products/products.module.ts (269 bytes)
CREATE src/products/products.service.spec.ts (474 bytes)
CREATE src/products/products.service.ts (651 bytes)
CREATE src/products/dto/create-product.dto.ts (33 bytes)
CREATE src/products/dto/update-product.dto.ts (181 bytes)
CREATE src/products/entities/product.entity.ts (24 bytes)
UPDATE src/app.module.ts (507 bytes)
```

Al finalizar se creará en la jerarquía del proyecto una carpeta con el nombre elegido y dentro se crearon los DTO con los que se manejarán las peticiones. Además se creó el controller, service y module del objeto.



La estructura que crea el comando integra los DTO'S, los cuales son la estructura de lo que la API espera recibir, comenzaremos a definir cada una de las clases products.



Un producto tiene: **name**, **code**, **stock**, **price**, **description**, por tanto, el DTO debe indicar que un producto debe tener esa estructura para ser creado, **pero** también la estructura que debe tener para modificarse. Porque estamos de acuerdo en que no puedo modificar el **code** por ejemplo, pero.... Si se puede modificar el **price**, **stock** por ejemplo, entonces se deben crear 2 DTO uno para crear producto y otro para modificarlo.

Comenzamos con la modificación del archivo **create-product.dto.ts**

TS **create-product.dto.ts** 5, U X

product-api > src > products > dto > TS create-prod

```
1  export class CreateProductDto {
2    name: string;
3    code: string;
4    stock: number;
5    price: number;
6    description: string;
7  }
8
```

En el archivo declaramos que propiedades tendrá el objeto **product** para poder ser creado. En el archivo **update-product.dto.ts** declaramos las propiedades que pueden o no ser modificadas del objeto. Si analizamos el archivo encontramos que:

TS **update-product.dto.ts** U X

product-api > src > products > dto > TS update-product.dto.ts > UpdateProductDto

```
1  import { PartialType } from '@nestjs/mapped-types';
2  import { CreateProductDto } from './create-product.dto';
3
4  export class UpdateProductDto extends PartialType(CreateProductDto) {
5
6  }
7
```

Crea automáticamente todos los atributos del objeto DTO, pero como opcionales

Usando los DTO en el Controller

Es importante analizar que nos creó el framework en el archivo **products.controller.ts** desde el principio. Se ha creado el **CRUD** básico de la API usando los DTO. Los DTO se usan



para recibir el cuerpo de la petición, validar la estructura y validar los datos incluyendo los tipos de datos, si son vacíos, si son nulos, etc etc.

```
update-product.dto.ts U  products.controller.ts U X
product-api > src > products > products.controller.ts > ...
1 import { Controller, Get, Post, Body, Patch, Param, Delete } from '@nestjs/common';
2 import { ProductsService } from '../products.service';
3 import { CreateProductDto } from '../dto/create-product.dto';
4 import { UpdateProductDto } from '../dto/update-product.dto';
5
6 @Controller('products')
7 export class ProductsController {
8   constructor(private readonly productsService: ProductsService) {}
9
10  @Post()
11  create(@Body() createProductDto: CreateProductDto) {
12    | return this.productsService.create(createProductDto);
13  }
14
15  @Get()
16  findAll() {
17    | return this.productsService.findAll();
18  }
19
20  @Get('/:id')
21  findOne(@Param('id') id: string) {
22    | return this.productsService.findOne(+id);
23  }
24
25  @Patch('/:id')
26  update(@Param('id') id: string, @Body() updateProductDto: UpdateProductDto) {
27    | return this.productsService.update(+id, updateProductDto);
28  }
29
30  @Delete('/:id')
31  remove(@Param('id') id: string) {
32    | return this.productsService.remove(+id);
33  }
34 }
35
```

Modificando los Service

Hasta este momento tenemos listo el controlador y el DTO, sin embargo hace falta implementar la lógica del service, es decir, la forma en cómo se manejan los datos. Si bien el framework ya me creo el service y me creo los métodos pero estos simplemente devuelven mensajes ilustrativos, veamos... Abrimos el archivo **products.service.ts**



```
products.service.ts U X
product-api > src > products > products.service.ts > ...
1 import { Injectable } from '@nestjs/common';
2 import { CreateProductDto } from '../dto/create-product.dto';
3 import { UpdateProductDto } from '../dto/update-product.dto';
4
5 @Injectable()
6 export class ProductsService {
7   create(createProductDto: CreateProductDto) {
8     return 'This action adds a new product';
9   }
10
11   findAll() {
12     return 'This action returns all products';
13   }
14
15   findOne(id: number) {
16     return `This action returns a ${id} product`;
17   }
18
19   update(id: number, updateProductDto: UpdateProductDto) {
20     return `This action updates a ${id} product`;
21   }
22
23   remove(id: number) {
24     return `This action removes a ${id} product`;
25   }
26 }
```

Como podemos observar, el framework nos creó la estructura del service pero no maneja los datos reales, solo implementó los métodos y devuelve un mensaje representativo. Ahora modificaremos el **service** para que manipule los datos reales del objeto **DTO**.

Modificando el service

Antes que nada, debemos crear la estructura que va a tener nuestro objeto **Product**, si bien tenemos el DTO no debemos confundir, el DTO sirve para recibir datos y la estructura del objeto es para el uso dentro del sistema. Para ello modificamos el archivo **producto.entity.ts**

```
product-api > src > products > entity.ts > ...
1 export class Product {
2   id: number;
3   name: string;
4   description: string;
5   price: number;
6   stock: number;
7 }
```

En el archivo service comenzaremos por declarar un arreglo que tendrá los datos en memoria, esto se hace de manera global NO dentro de algún método.



```
-
6  @Injectable()
7  export class ProductService {
8
9  private products:Product[]=[]
10
```

En el método `create`, modificaremos para que cree un producto y lo agregue al arreglo, tomando en cuenta que el **DTO** llega desde el **front**.

```
9
10 create(createProductDto: CreateProductDto) {
11     const producto: Product = {
12         id: Date.now(),
13         ...createProductDto,
14     };
15
16     this.products.push(producto);
17     return 'El objeto ah sido creado exitosamente';
18 }
```

Para probar la api basta con abrir postman la url **localhost:3000/productos** la petición es de tipo **Post** y en el cuerpo de la petición colocar el json del producto

```
{
  "name": "Laptop",
  "description": "Ryzen 7",
  "price": 15000,
  "stock": 5,
  "category": "Computadoras"
}
```

El mensaje de respuesta debe ser el que se colocó en el Service.

Modificando el metodo `findAll()` del service

En el metodo debe devolver todos los registros del arreglo, en este caso basta con indicarle que retorne el arreglo.

```
20 findAll() {
21     return this.products;
22 }
23
```




Para probar la api basta con abrir postman la url **localhost:3000/productos** la petición es de tipo **GET**. Si se han agregado datos deberá mostrar la lista de objetos que se crearon, si no, agregar primero datos luego probar esta api.

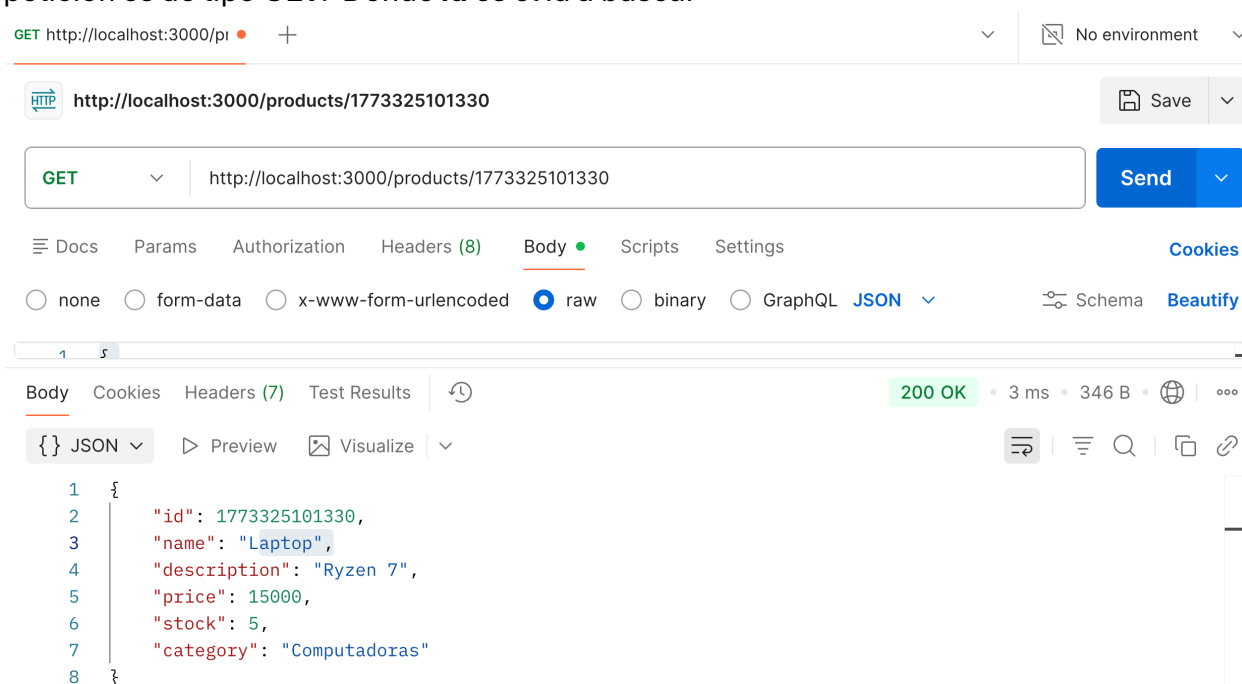
NOTA: cada vez que la aplicaicon se compila se borra los datos del arreglo

Modificando el metodo findOne

Este metodo debe devolver solo un registro basado en id.

```
23  
24   findOne(id: number) {  
25     return this.products.find((product) => product.id === id);  
26   }  
27
```

Para probar la api basta con abrir postman la url **localhost:3000/productos/<id>** la petición es de tipo **GET**. Donde **id** es el id a buscar



Modificando el metodo update

El metodo update permite modificar las propiedades del objeto en memoria. Utiliza el dto de actualizar y recibe en el cuerpo de la petición el json con los datos a modificar.

En postman usar el tipo de petición **PATCH**.



```
28 actualizar(id: number, updateProductDto: UpdateProductDto) {
29   const product = this.products.find((p) => p.id === id);
30
31   if (!product) {
32     return 'objeto no localizado';
33   }
34   Object.assign(product, updateProductDto);
35
36   return 'Objeto actualizado';
37 }
38 remove(id: number) {
39   return `This action removes a #${id} product`;
40 }
```

Actividad

Crear los siguientes endpoints

1. Buscar por nombre
2. Listar los productos que tengan stock disponible
3. Buscar por rango de precio
4. Restar stock a un producto